

MLOps – Assignment Report

Heart Disease Prediction API

Name: Bhooshaan A

ID: 2025cs05032

Demo Recording Link:

<https://drive.google.com/file/d/1waLydlMx2il6unlLuzCD0gHqu7y6XuNH/view?usp=sharing>

GitHub Repo Link:

<https://github.com/Bhooshaan-2025cs05032/MLOps-Assignment>

1. Project Overview

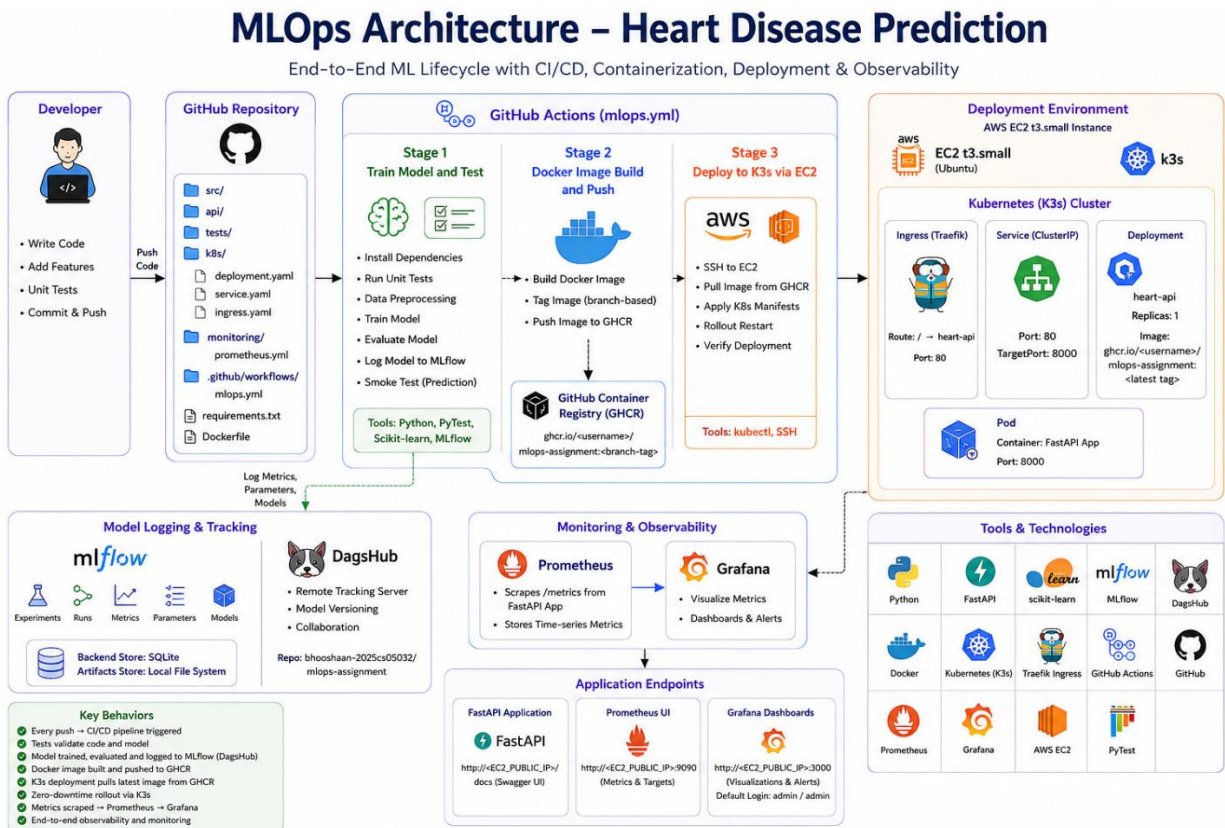
This project demonstrates a complete End-to-End MLOps workflow for a Heart Disease Prediction system. The project includes:

- FastAPI-based ML inference API
- CI/CD using GitHub Actions
- Docker containerization
- Kubernetes deployment using K3s
- Monitoring using Prometheus and Grafana
- ML experiment tracking using MLflow + DagsHub
- Image Store with GitHub Container Registry (GHCR)
- Ingress-based application exposure

2. Tech Stack

Category	Technology
Programming Language	Python
API Framework	FastAPI
ML Library	Scikit-learn
Experiment Tracking	MLflow
Remote Tracking	DagsHub
Containerization	Docker
CI/CD	GitHub Actions
Container Registry	GitHub Container Registry (GHCR)
Orchestration	Kubernetes (K3s)
Monitoring	Prometheus
Visualization	Grafana
Cloud	AWS EC2
Ingress	Traefik

3. Architecture Diagram of the E2E MLOps Pipeline



4. Setup/Install Instructions Guide

4.1. Local Project Setup

Clone Repository

```
git clone <YOUR_REPOSITORY_URL>
cd <REPOSITORY_NAME>
```

REPOSITORY_URL: <https://github.com/Bhooshaan-2025cs05032/MLOps-Assignment.git>

Create Virtual Environment

```
python3 -m venv mlopsenv
```

Activate Virtual Environment

Linux / Mac

```
source mlopsenv/bin/activate
```

Windows

```
mlopsenv\Scripts\activate
```

Install Dependencies

```
pip install -r requirements.txt
```

4.2. Run Training Pipeline

Configure MLflow / DagsHub Environment Variables

```
export MLFLOW_TRACKING_URI=<YOUR_MLFLOW_URI>  
export MLFLOW_TRACKING_USERNAME=<YOUR_USERNAME>  
export MLFLOW_TRACKING_PASSWORD=<YOUR_PASSWORD>
```

Run Data Processing

```
python src/data_processing.py
```

Train Model

```
python src/train_model_and_log.py
```

4.3. Run API Locally

Start FastAPI Application

```
uvicorn api.app:app --host 0.0.0.0 --port 8000
```

Access Swagger UI

```
http://localhost:8000/docs
```

4.4. Run Tests

Run Unit Tests

```
pytest tests/
```

Run Coverage

```
pytest tests/ \
--cov \
--cov-config=.coveragerc \
--cov-report=term-missing
```

4.5. Docker Setup

Build Docker Image

```
docker build -t heart-api .
```

Run Docker Container

```
docker run -p 8000:8000 \
-e MLFLOW_TRACKING_URI=<URI> \
-e MLFLOW_TRACKING_USERNAME=<USERNAME> \
-e MLFLOW_TRACKING_PASSWORD=<PASSWORD> \
heart-api
```

Verify API

```
http://localhost:8000/docs
```

4.6. AWS EC2 Setup

Create EC2 Instance

Recommended Configuration:

Property	Value
Instance Type	t3.small
OS	Ubuntu
Storage	20 GB

Open Security Group Ports

Port	Purpose
22	SSH
80	Ingress / API
3000	Grafana
9090	Prometheus

4.7. Install Docker on EC2

```
curl -fsSL https://get.docker.com | sh
```

Add User to Docker Group

```
sudo usermod -aG docker ubuntu  
newgrp docker
```

4.8. Install K3s Kubernetes

```
curl -sfl https://get.k3s.io | sh -
```

Configure kubectl Access

```
mkdir -p ~/.kube  
  
sudo cp /etc/rancher/k3s/k3s.yaml ~/.kube/config  
  
sudo chown ubuntu:ubuntu ~/.kube/config
```

Verify Kubernetes

```
kubectl get nodes
```

4.9. Kubernetes Deployment

Create Kubernetes Directory

```
mkdir ~/k8s
```

Kubernetes Manifests

Place the following files inside:

```
~/k8s/  
├── deployment.yaml  
├── service.yaml  
└── ingress.yaml
```

Apply Manifests

```
kubectl apply -f ~/k8s/deployment.yaml  
kubectl apply -f ~/k8s/service.yaml  
kubectl apply -f ~/k8s/ingress.yaml
```

Verify Pods

```
kubectl get pods
```

Verify Ingress

```
kubectl get ingress
```

4.10. Prometheus Setup

Create Monitoring Directories

```
mkdir ~/monitoring  
mkdir ~/prometheus_data
```

Create Prometheus Config

File:

```
~/monitoring/prometheus.yml
```

Content:

```
global:  
  scrape_interval: 30s  
  
scrape_configs:  
  - job_name: "heart-api-k8s"  
    metrics_path: /metrics  
    static_configs:  
      - targets: ["localhost"]
```

Run Prometheus

```
docker run -d \  
  --name prometheus \  
  --restart always \  
  --network host \  
  -v /home/ubuntu/monitoring/prometheus.yml:/etc/prometheus/prometheus.yml \  
  -v /home/ubuntu/prometheus_data:/prometheus \  
  prom/prometheus
```

Access Prometheus

`http://<EC2_PUBLIC_IP>:9090`

4.11. Grafana Setup

Run Grafana

```
docker run -d \
  --name grafana \
  --restart always \
  -p 3000:3000 \
  grafana/grafana
```

Access Grafana

`http://<EC2_PUBLIC_IP>:3000`

Can create dashboards of your choice with queries for metrics

Default Credentials

Username: admin

Password: admin

Add Prometheus Data Source

`http://<EC2_PUBLIC_IP>:9090`

4.12. GitHub Secrets Setup

Configure the following GitHub Secrets:

Secret Name

EC2_HOST

EC2_SSH_KEY

MLFLOW_TRACKING_URI

MLFLOW_TRACKING_USERNAME

MLFLOW_TRACKING_PASSWORD

4.13. CI/CD Pipeline Overview

On code commit, the GitHub Actions workflow performs:

1. Run unit tests
 2. Run preprocessing
 3. Train model
 4. Validate model artifact
 5. Build Docker image
 6. Push image to GHCR
 7. SSH into EC2
 8. Create/update Kubernetes secrets
 9. Apply Kubernetes manifests
 10. Perform rolling deployment
-

4.14. Deployment Verification

Verify Deployment

```
kubectl get deployments
```

Verify Pods

```
kubectl get pods
```

Verify Current Image

```
kubectl get deployment heart-api \
-o=jsonpath='{$.spec.template.spec.containers[0].image}'
```

Verify Rollout Status

```
kubectl rollout status deployment heart-api
```

4.15. Application Endpoints

Service	URL
FastAPI Swagger UI	http://<PUBLIC_IP or ELASTIC_IP>/docs
Prometheus	http:// :<PUBLIC_IP or ELASTIC_IP>/:9090
Grafana	http:// :<PUBLIC_IP or ELASTIC_IP>/:3000

5. Exploratory Data Analysis (EDA) and Modelling Choices

Dataset Understanding

The project uses a Heart Disease Prediction dataset containing clinical and medical attributes such as:

- Age
- Sex
- Chest Pain Type (cp)
- Resting Blood Pressure (trestbps)
- Cholesterol (chol)
- Fasting Blood Sugar (fbs)
- Resting ECG (restecg)
- Maximum Heart Rate (thalach)
- Exercise Induced Angina (exang)
- ST Depression (oldpeak)
- Slope
- Number of Major Vessels (ca)
- Thalassemia (thal)

The target variable represents the presence or absence of heart disease.

EDA Performed

1. Data Inspection

Initial analysis included:

- Checking dataset shape and schema
- Verifying missing values
- Inspecting feature distributions
- Identifying categorical and numerical columns

```
df.info()
✓ 0.0s

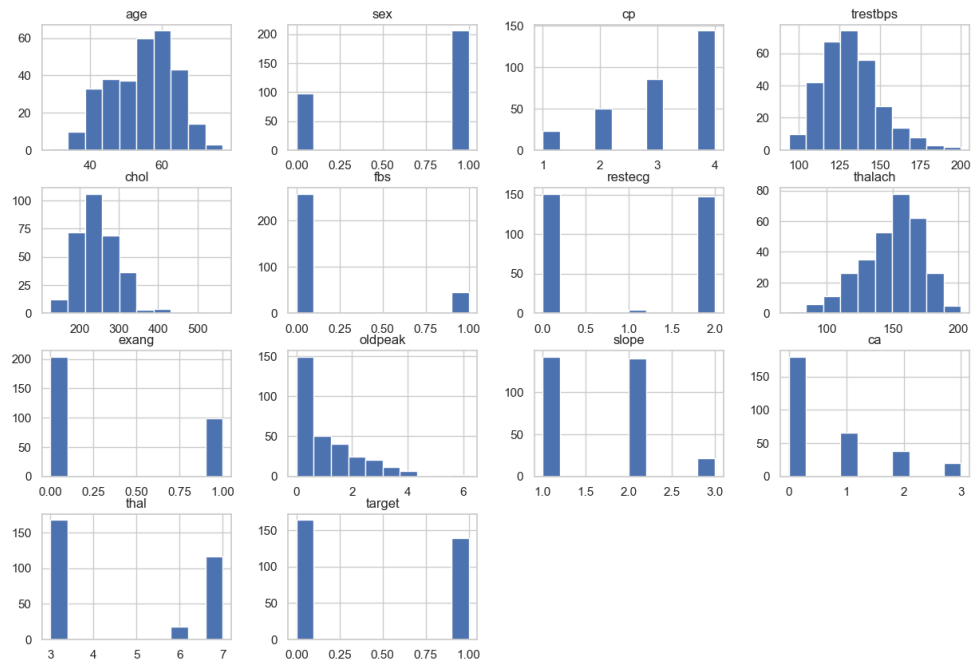
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 303 entries, 0 to 302
Data columns (total 14 columns):
#   Column      Non-Null Count  Dtype
---  -
0   age         303 non-null   int64
1   sex         303 non-null   int64
2   cp          303 non-null   int64
3   trestbps    303 non-null   int64
4   chol        303 non-null   int64
5   fbs         303 non-null   int64
6   restecg     303 non-null   int64
7   thalach     303 non-null   int64
8   exang       303 non-null   int64
9   oldpeak     303 non-null   float64
10  slope       303 non-null   int64
11  ca          299 non-null   float64
12  thal        301 non-null   float64
13  target      303 non-null   int64
dtypes: float64(3), int64(11)
memory usage: 33.3 KB
```

2. Statistical Analysis

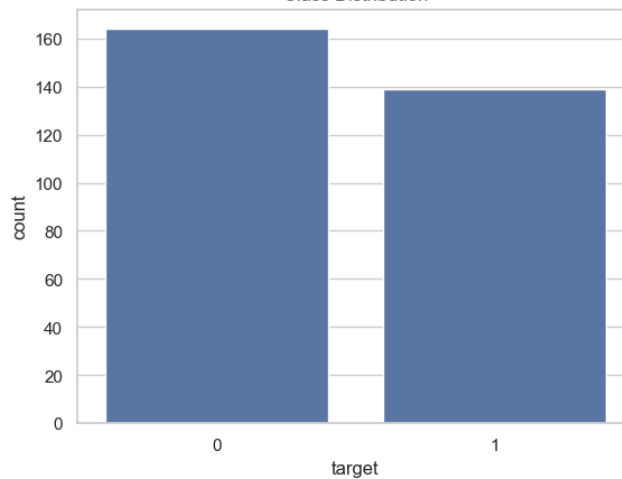
Descriptive statistics were generated to understand:

- Mean
- Median
- Standard deviation
- Feature ranges
- Class balance

Feature Distributions



Class Distribution



This helped identify potential outliers and understand feature spread.

3. Correlation Analysis

Correlation analysis was performed to:

- Understand relationships between features
- Identify highly influential variables
- Detect multicollinearity

Features such as chest pain type, maximum heart rate, exercise-induced angina, and ST depression showed meaningful correlation with the target.

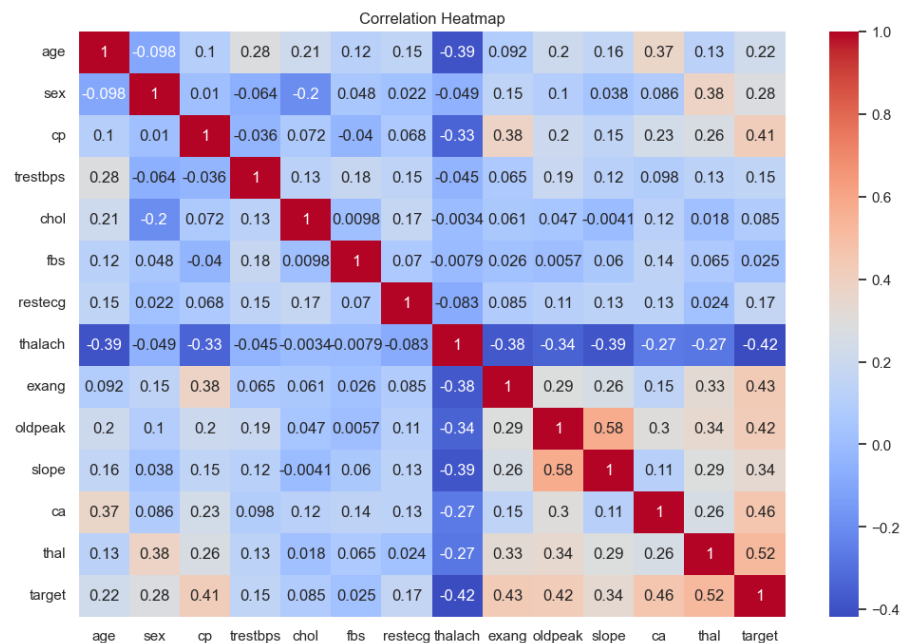
4. Visualization

EDA visualizations included:

- Histograms
- Count plots
- Box plots
- Correlation heatmaps

These helped analyze:

- Feature distributions
- Outliers
- Target imbalance
- Feature importance trends



Data Preprocessing

The preprocessing pipeline included:

- Handling missing values (if any)
- Feature selection
- Data cleaning
- Train-test split
- Scaling / normalization where applicable

The preprocessing logic was modularized into a dedicated pipeline for reproducibility.

Modelling Choices

Selected Model

The project uses a supervised machine learning classification approach (Logistic Regression and Random Forest) for predicting heart disease.

Multiple classical ML algorithms were evaluated, with emphasis on:

- Interpretability
 - Simplicity
 - Performance on tabular medical data
 - Faster inference
 - Lower infrastructure requirements
-

Why Classical ML Instead of Deep Learning?

Deep learning was intentionally avoided because:

- Dataset size is relatively small
- Tabular healthcare data often performs well with classical ML
- Simpler models are easier to interpret
- Lower computational overhead
- Faster training and inference

This aligns well with production-friendly MLOps deployment requirements.

Evaluation Metrics

The model evaluation considered metrics such as:

- Accuracy
- Precision
- Recall
- F1 Score
- Confusion Matrix

Special importance was given to recall and F1-score due to the healthcare nature of the problem.

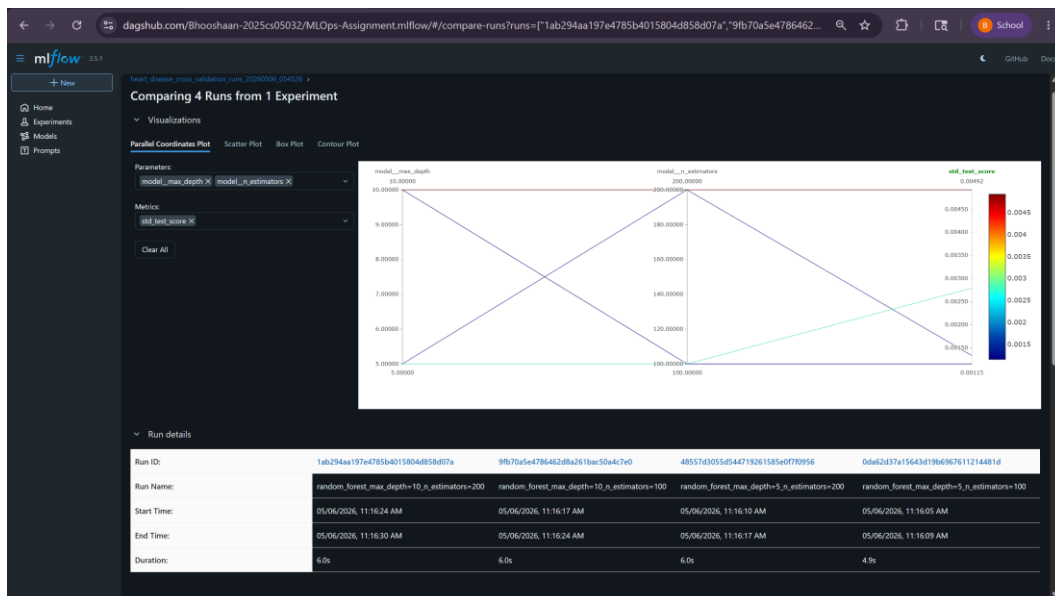
6. Model Tracking with MLflow – Experiment Tracking Summary

MLflow was used to:

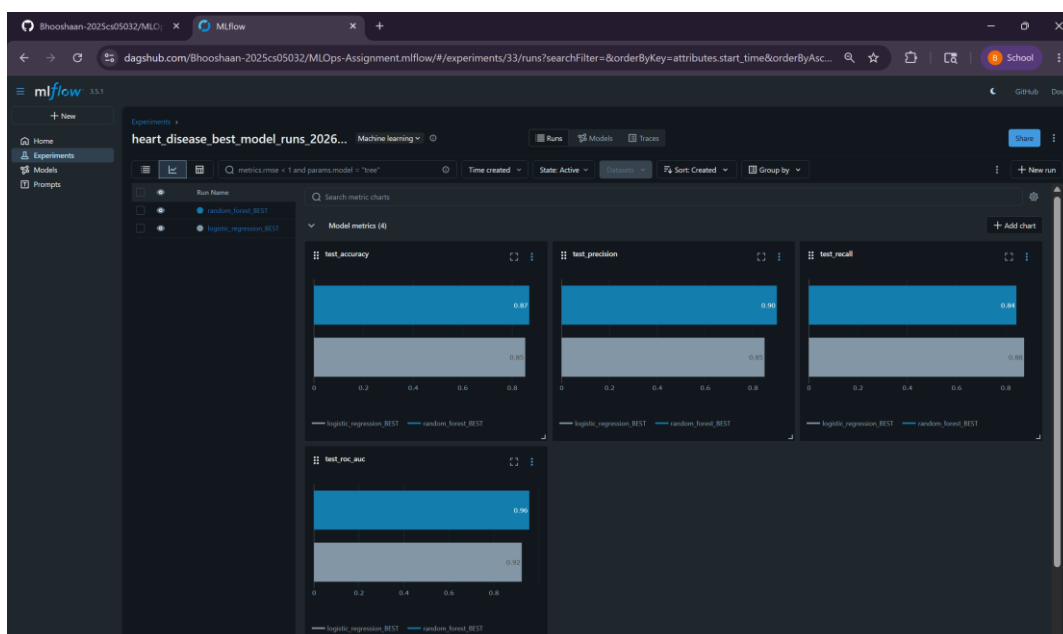
- Track experiments
- Log hyperparameters
- Log metrics
- Store trained models
- Maintain experiment reproducibility

DagsHub was integrated as the remote MLflow tracking backend.

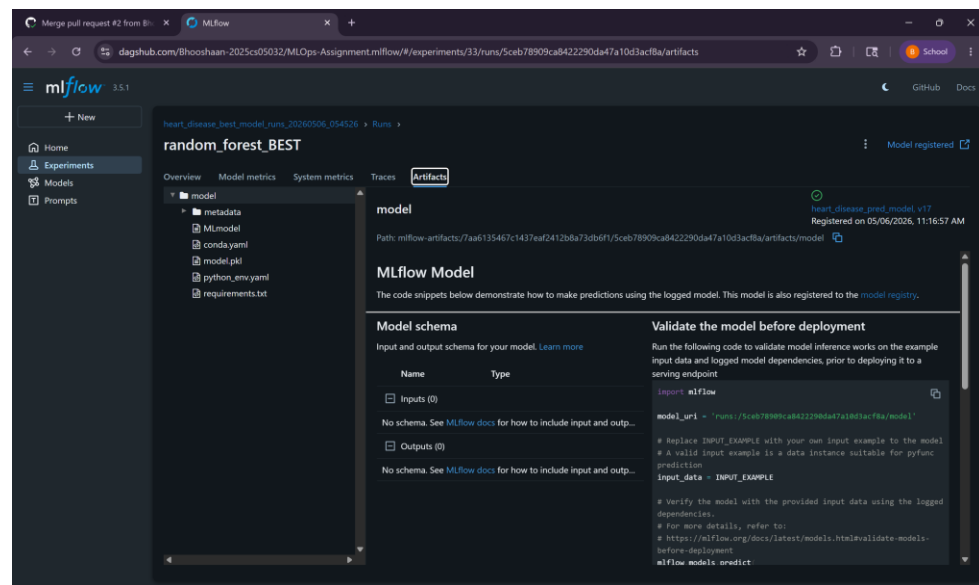
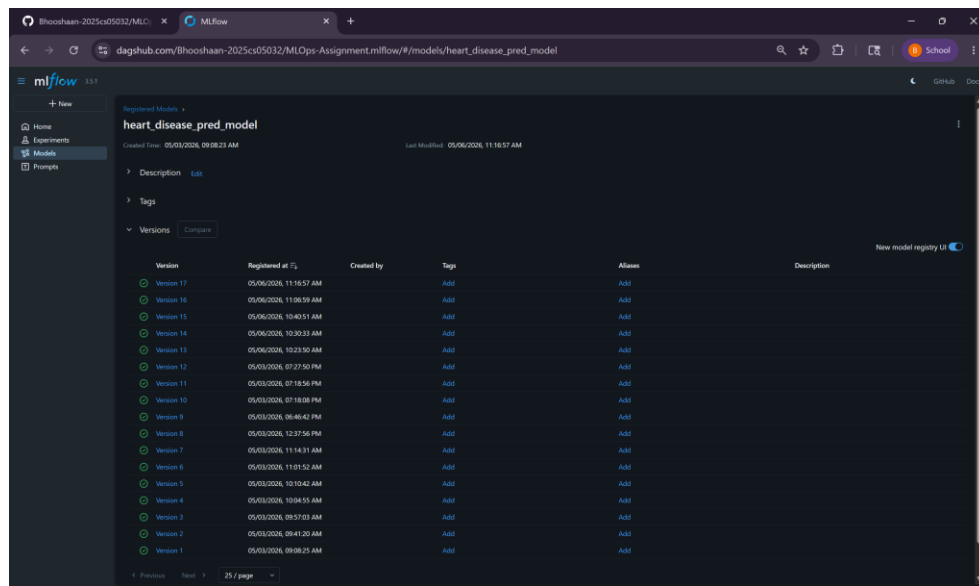
- Cross validation runs for hyperparameter tuning was logged for each model



- For each model best hyperparameter version was logged for test metrics.



- Best model with high AUC-ROC was registered in the model registry (Every training registers the model with updated version) – Also Model Artifacts with env recreation files stored.



7. CI CD Deployment Workflow

- Github Actions was used for CI/CD Pipeline

- Phase 1: Build-Train-Test: Fetches data, installs requirements in a clean environment, runs unit tests and fails if coverage is less than 80%, pre-processes and cleans data, trains model and logs the artifacts to DagsHub-MLflow and registers best model.

The screenshot shows a GitHub Actions workflow run for the 'train-model-and-test' job. The workflow is part of the 'MLOps-Assignment' repository. The job status is 'succeeded' and it was completed 4 days ago. The workflow steps include: Set up job, Checkout code, Setup Python, Install dependencies, Run tests with coverage, Run data preprocessing, Train model, Verify model artifact, Smoke test prediction, Post Setup Python, Post Checkout code, and Complete job. The 'Run tests with coverage' step is highlighted, showing a success status.

The screenshot shows a GitHub Actions workflow run for the 'build-train-test' job. The workflow is part of the 'MLOps-Assignment' repository. The job status is 'failed' and it was completed last week. The workflow steps include: Run tests with coverage, Run data preprocessing, Train model, Check model file, Smoke test prediction, Post Set up Python, Post Checkout repository, and Complete job. The 'Run tests with coverage' step is highlighted, showing a failure status. The failure message indicates that the required test coverage of 80% was not reached, with a total coverage of 71.33%.

Name	Stats	Miss	Cover	Missing
api/app.py	40	0	100%	
api/model_load_predict.py	59	41	31%	61-73, 81-88, 110-115, 129-132, 164-182
src/data_processing.py	44	0	100%	
TOTAL	143	41	71%	

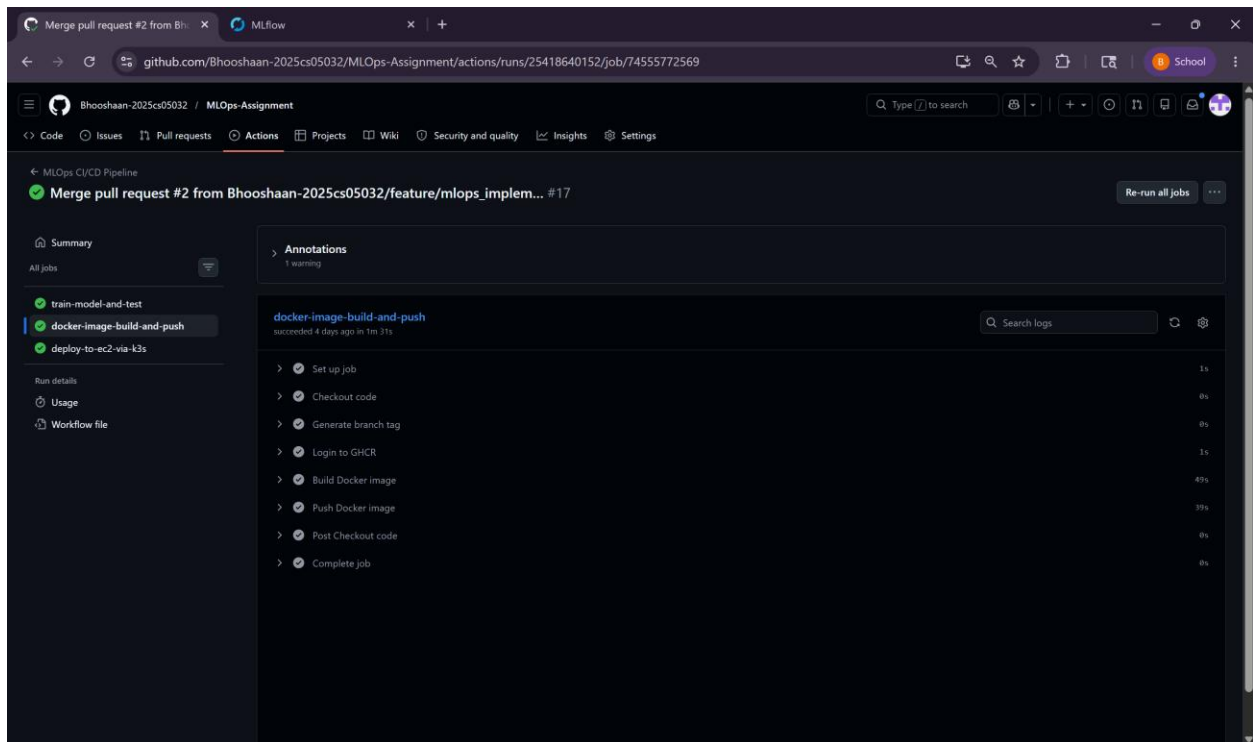
FAIL Required test coverage of 80% not reached. Total coverage: 71.33%

16 passed, 25 warnings in 2.65s

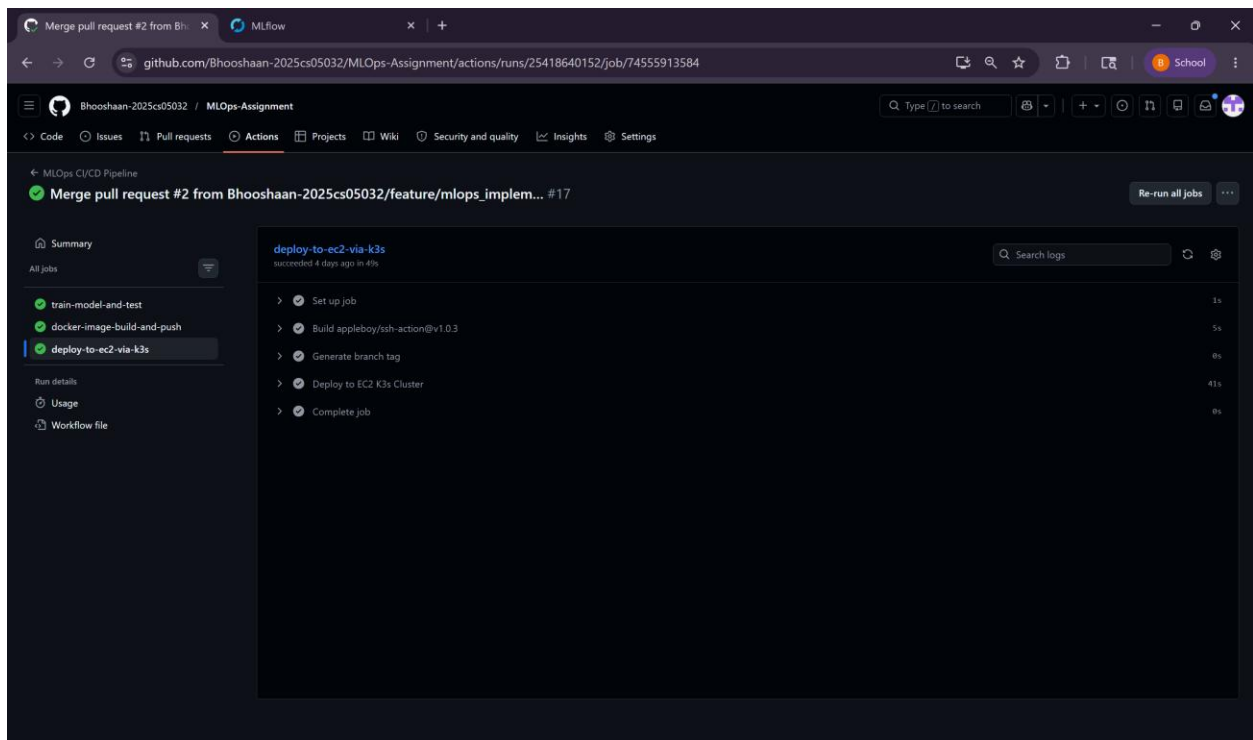
Error: Process completed with exit code 1.

(Tests failed as coverage is just 71.33%)

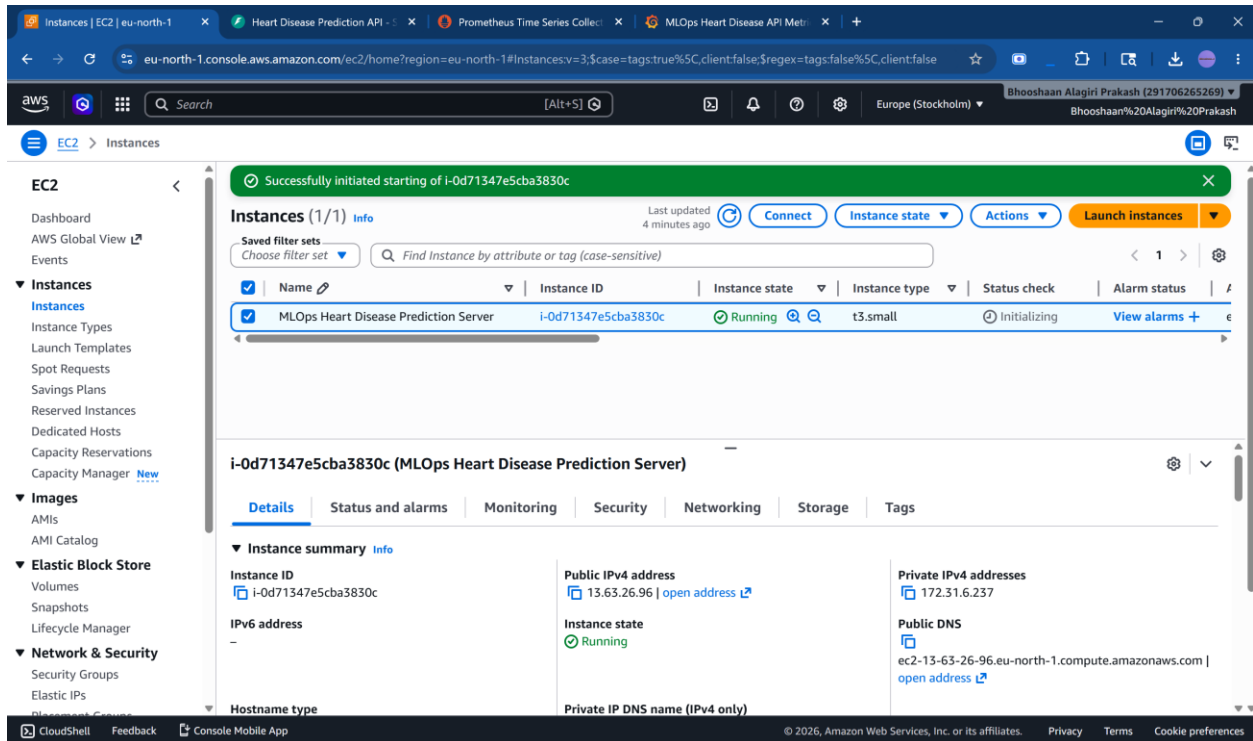
- Phase 2: Docker Image Build and Push: Fetches pkl file from DagsHub – MLflow Model registry, builds docker image and pushes it into Github Container Registry



- Phase 3: Deploy to EC2 via K3S: Pulls the latest image from GHCR and deploys in on K3S Pods



- AWS EC2 Instance t3.small Up and Running



- SSH into Instance and shows active deployment, active pod, kube rollout status successful and Prometheus and Grafana running.

```
C:\Bhooshaan_Local\BITS Local\Sem2\Assignments\MLOps\AWS EC2>ssh -i mlops-key.pem ubuntu@13.63.26.96
Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0.0-1004-aws x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Sun May 10 08:15:26 UTC 2026

System load:  0.1          Temperature:   -273.1 C
Usage of /:   55.3% of 18.25GB  Processes:    145
Memory usage: 62%          Users logged in: 0
Swap usage:   0%            IPv4 address for ens5: 172.31.6.237

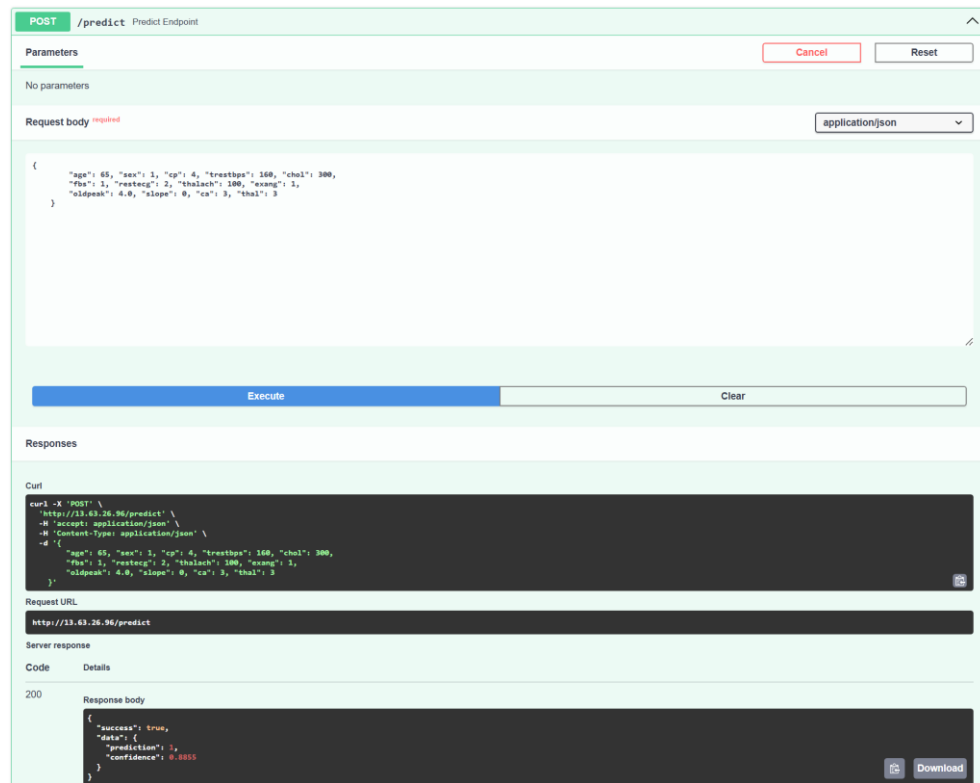
Expanded Security Maintenance for Applications is not enabled.

3 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

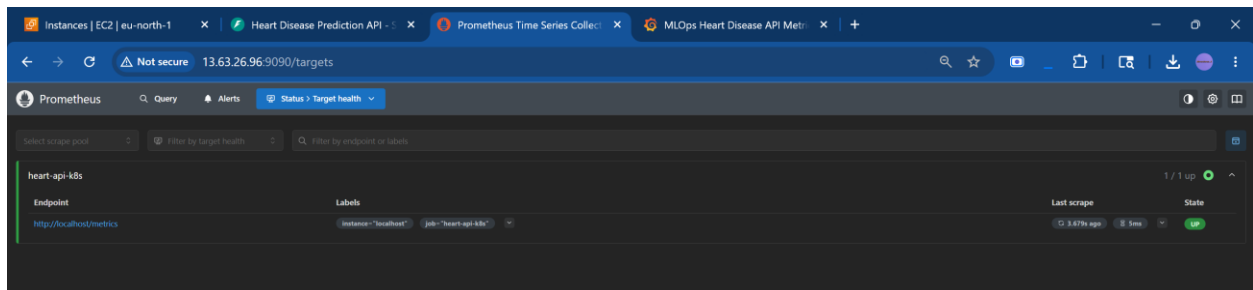
Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Last login: Sun May 10 08:13:56 2026 from 49.47.218.104
ubuntu@ip-172-31-6-237:~$ kubectl get deployments
NAME      READY   UP-TO-DATE   AVAILABLE   AGE
heart-api 1/1      1            1           4d4h
ubuntu@ip-172-31-6-237:~$ kubectl get pods
NAME      READY   STATUS    RESTARTS   AGE
heart-api-7cb48ff5f6-6jwck 1/1      Running    1 (4d2h ago) 4d2h
ubuntu@ip-172-31-6-237:~$ kubectl get deployment heart-api -o=jsonpath='{$.spec.template.spec.containers[0].image}'
ghcr.io/bhooshaan-2025cs05032/mlops-assignment:mainubuntu@ip-172-31-6-237:~$ kubectl rollout status deployment heart-api
deployment "heart-api" successfully rolled out
ubuntu@ip-172-31-6-237:~$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED    STATUS    PORTS
9574b46e6850   prom/prometheus "/bin/prometheus --c..." 4 days ago Up 10 minutes 0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
44409882ea51   grafana/grafana "/run.sh"               6 days ago Up 10 minutes
ubuntu@ip-172-31-6-237:~$
```

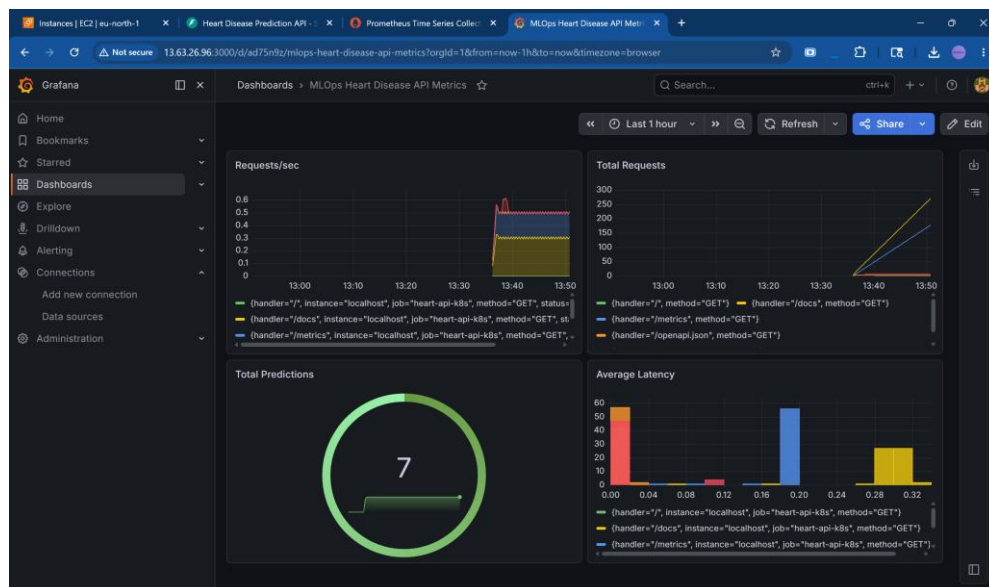
- Sample Prediction Made using the FastAPI Swagger UI



- Prometheus up and scraping the metrics endpoint



- Grafana Dashboard Up and Running used for monitoring metrics



The system supports automatic recovery after EC2 restart because:

- K3s runs as a system service
 - Kubernetes restores deployments automatically
 - API end-point exposed via Traefik Ingress
 - Prometheus uses Docker restart policy
 - Grafana uses Docker restart policy
 - Elastic IP maintains stable endpoint access (Elastic IP is created in EC2 mapping to the t3.small instance)
-

8. Key Features

- End-to-End MLOps workflow
 - Automated CI/CD pipeline
 - Kubernetes-based deployment
 - Ingress-based exposure
 - Branch-aware deployments
 - Monitoring and observability
 - ML experiment tracking
 - Containerized inference service
 - Rolling deployments
 - Secure secret management (Stored in GitHub Secrets)
-

9. Conclusion

This project demonstrates a production-style MLOps architecture covering:

- Model development
- CI/CD automation
- Containerization
- Kubernetes orchestration
- Monitoring and observability
- Secure deployment practices
- Cloud-native deployment

The implementation provides a strong foundation for scalable and maintainable machine learning systems.